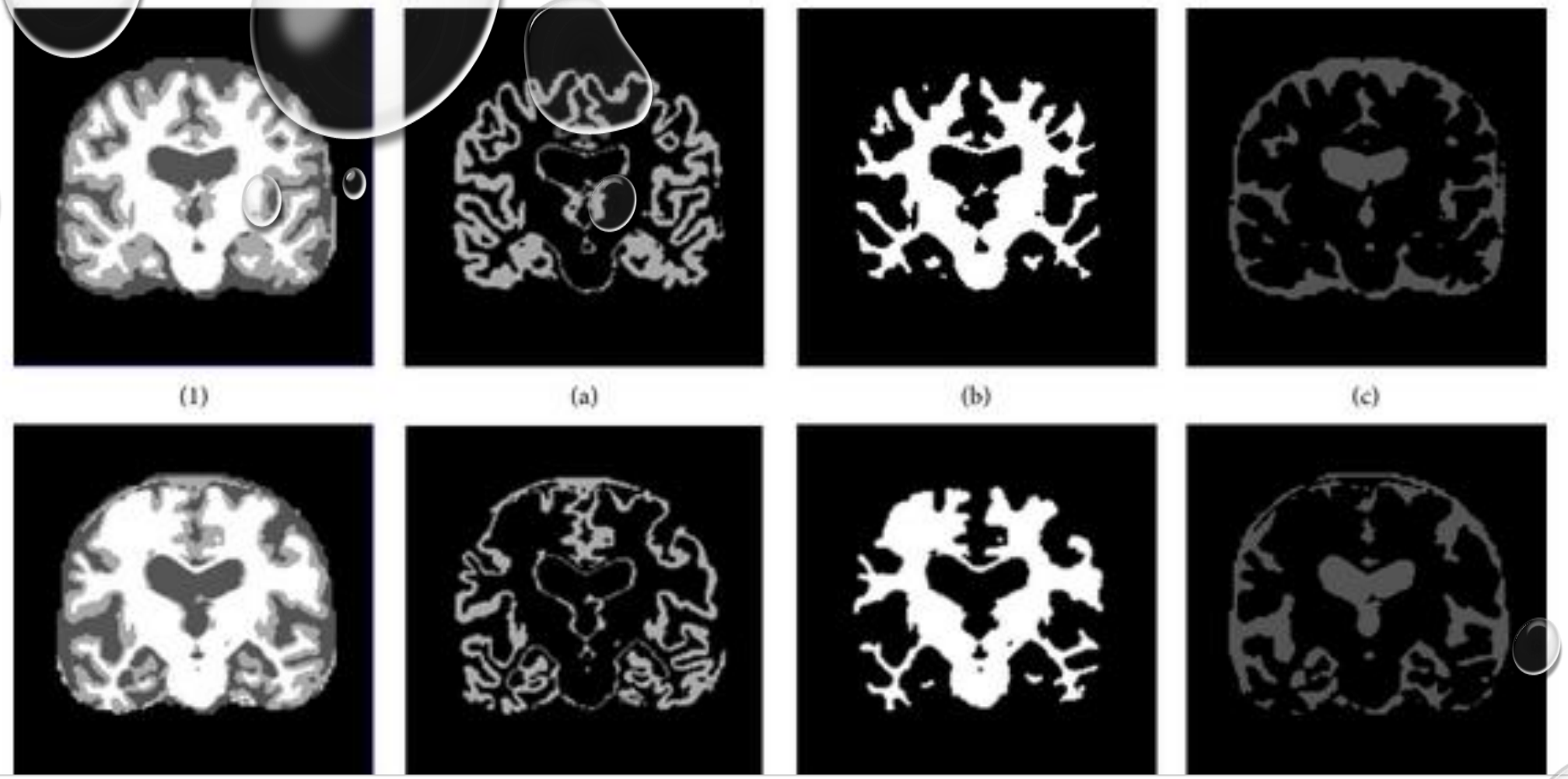
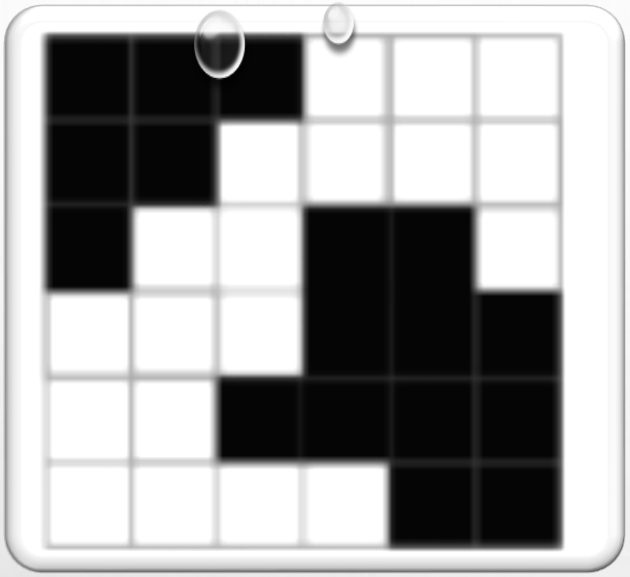
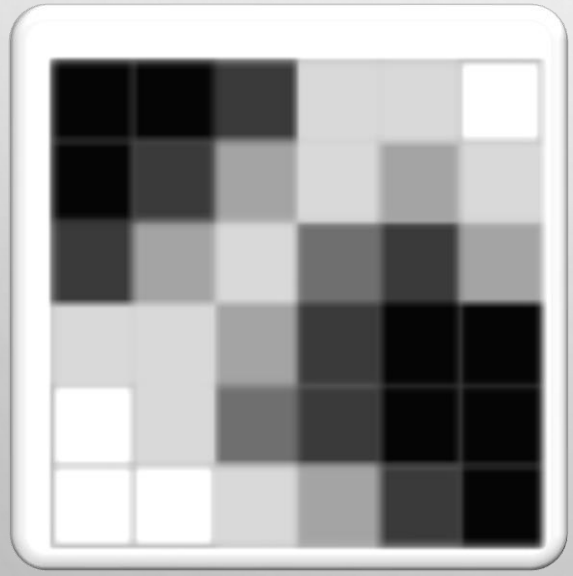


SEGMENTATION : REGIONS

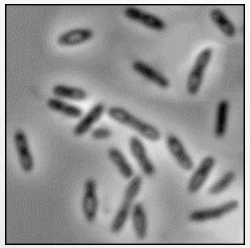




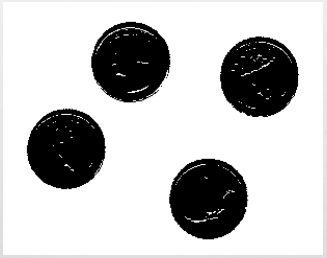
WHAT IS IMAGE SEGMENTATION

REGION-BASED SEGMENTATION

- Goal is to find objects of interest in an image

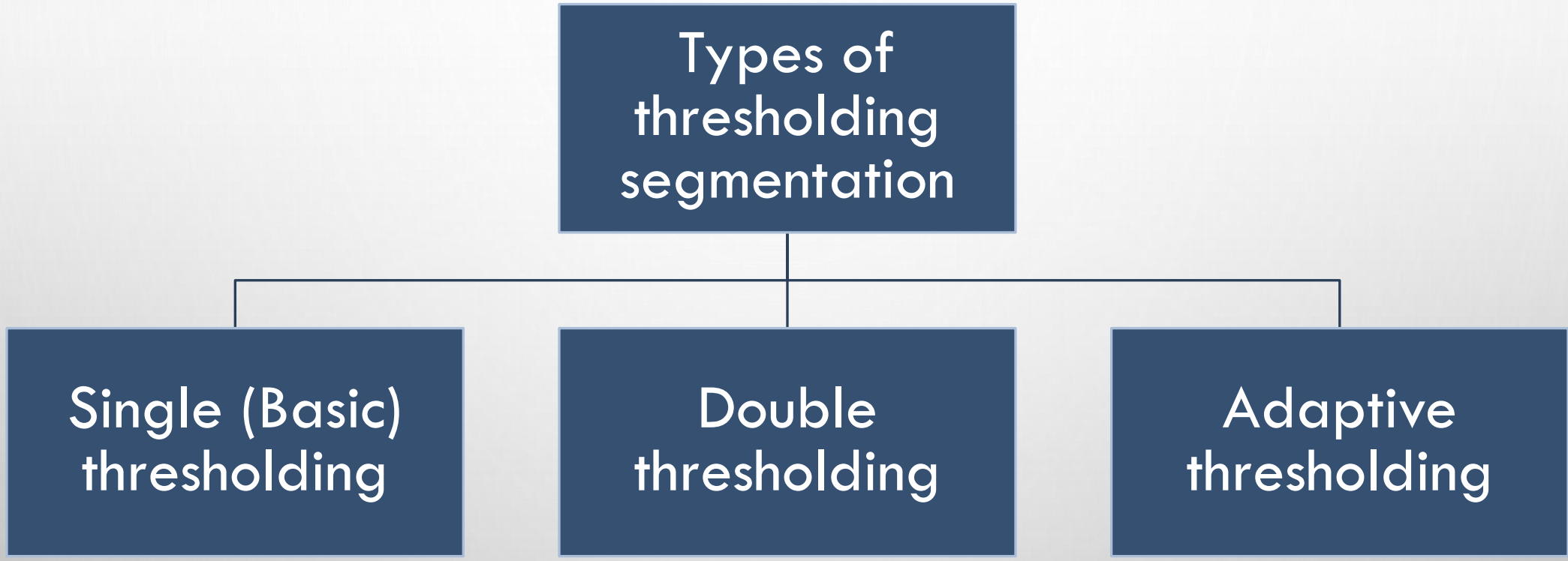







- Tons of ways to do this.
- Here we consider a methods based on thresholding and clustering

SEGMENTATION USING THRESHOLDING



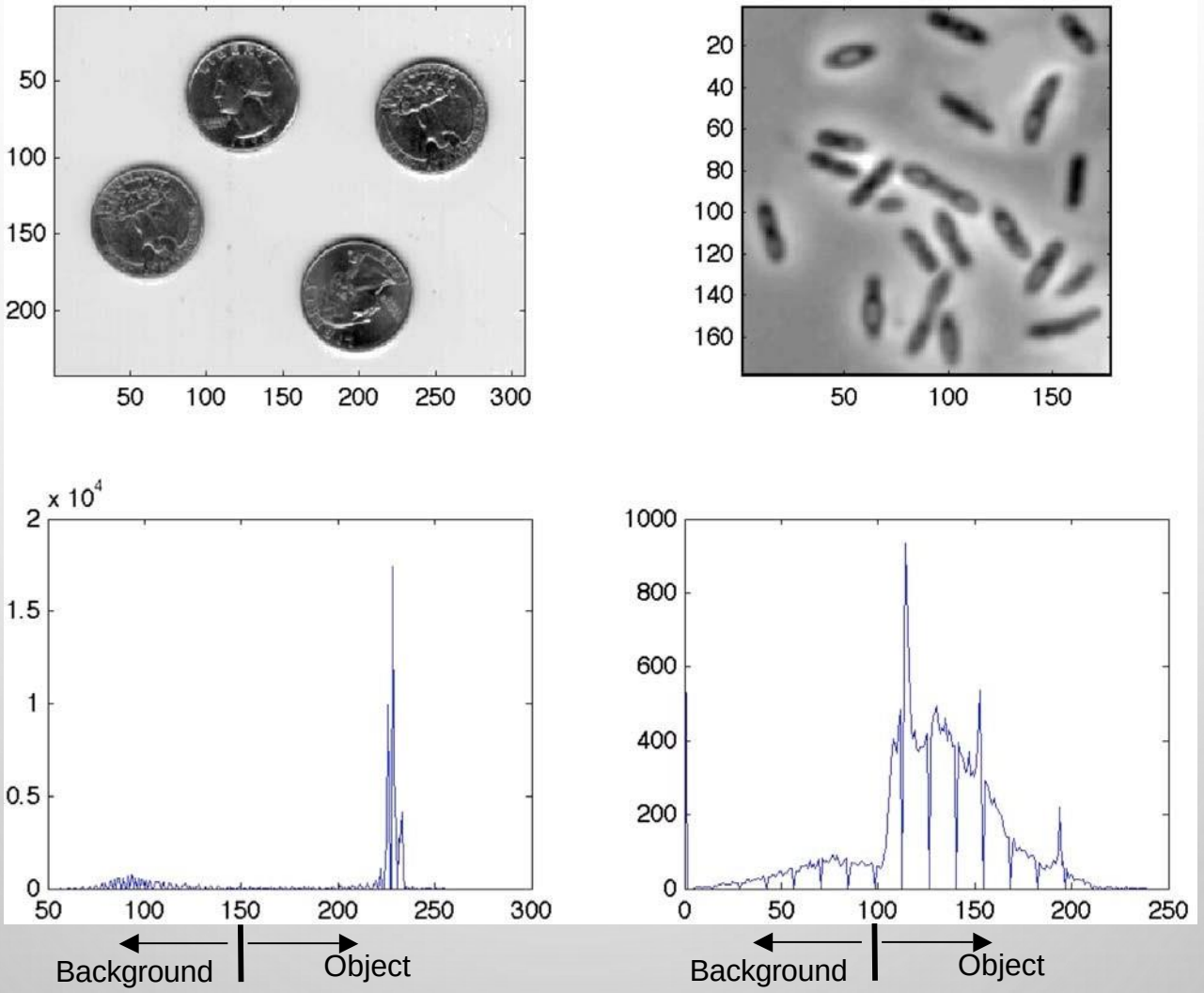
BASIC (SINGLE) THRESHOLDING

- Assume the image is comprised of two types (or *classes*) of pixels
  - Background: call this class 0
  - Object: call this class 1
- Segmentation = assigning a label, 0 or 1 to each pixel to indicate whether it is background or object
- Basic thresholding idea is to generate the label according to the rule (for dark object on light background)

$$l[m,n] = \begin{cases} 0 & f[m,n] \geq T \\ 1 & f[m,n] < T \end{cases}$$

T = threshold

THRESHOLD CHOICE USING HISTOGRAMS

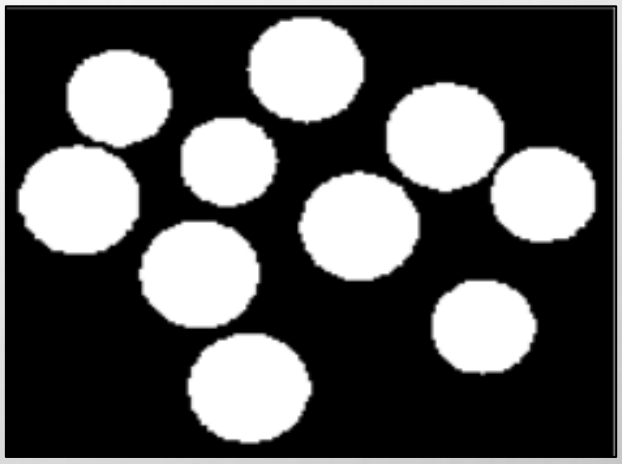


IMPLEMENTATION:

Suppose we have an 8 bit image, stored as the variable X. Then the command X>T will perform the thresholding. We can view the result with imshow. the commands

```
>> c=imread('coins.tif');
>>imshow(c),figure,imshow(c>110);
```





Thresholded image for coins

Matlab has the im2bw function, which thresholds an image of any data type, using the general syntax:

```
Im2bw(image, level)
```

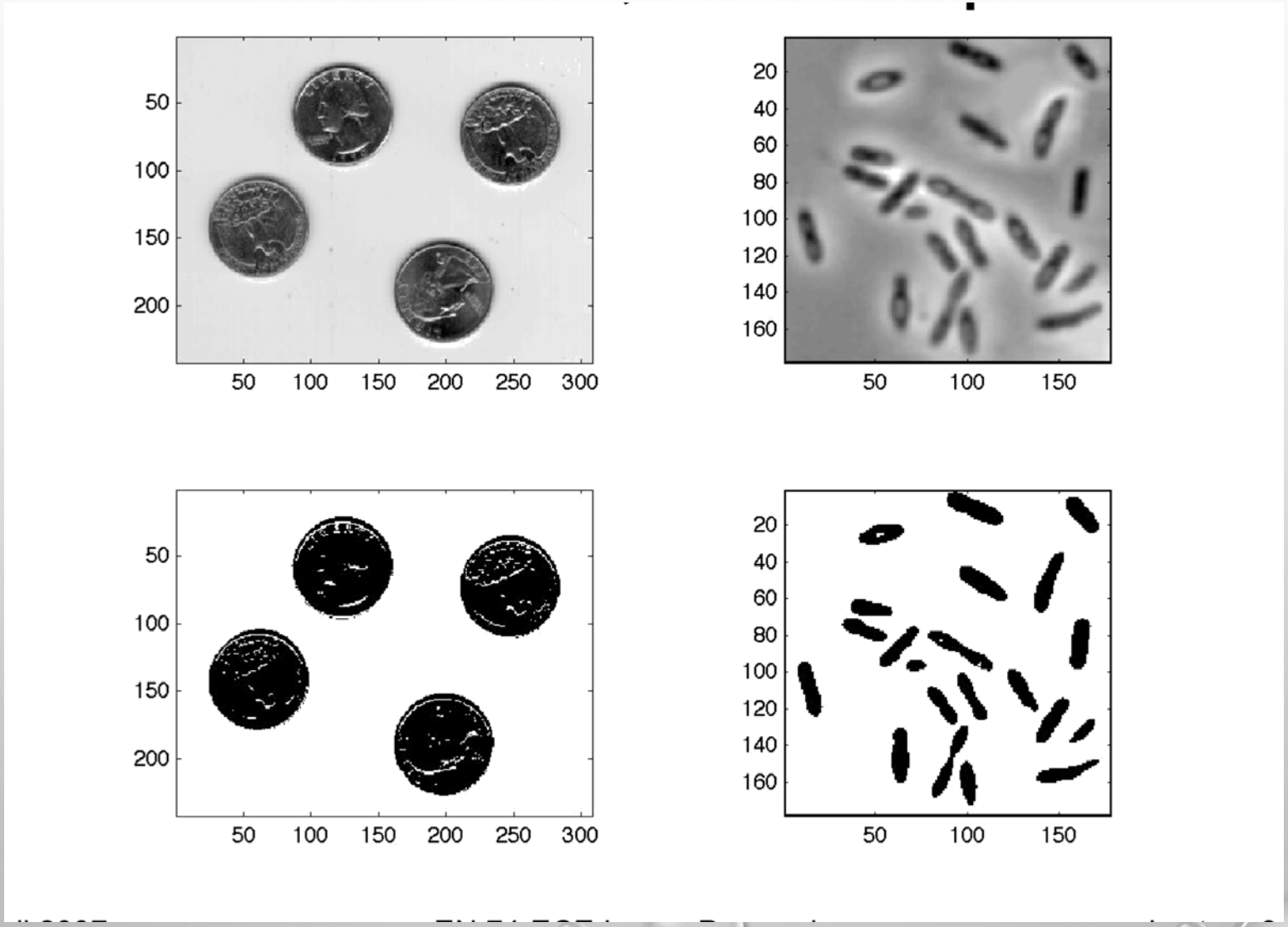
where level is a value between 0 and 1

Example:
the thresholded coins and circles images above could be obtained using

```
>> im2bw(c,0.53);
>> im2bw(i,0.34);
```

The im2bw function automatically scales the value level to a grey value appropriate to the image type, and then performs a thresholding by our first method.

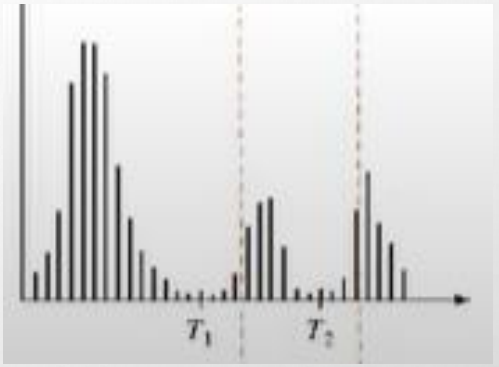
RESULTS: GOOD, BUT NOT PERFECT



Double thresholding :

Here we choose two values T1 and T2 and apply a thresholding operation as

A pixel becomes
- white if its grey level is between T1 and T2
- black if its grey level is otherwise



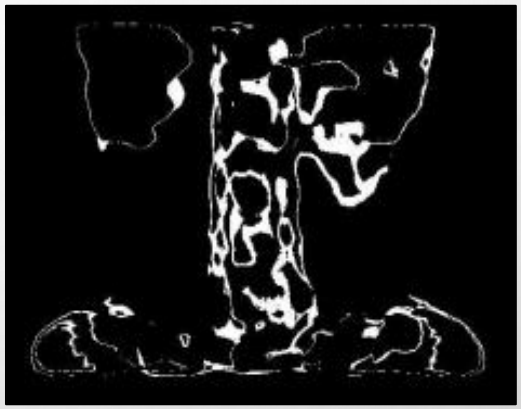
We can implement this by a simple variation on the above method

X>T1 & X<T2

Consider the following sequence of commands, which start by producing an 8-bit grey version of the indexed image spine.tif

```
>> [x,map]=imread('spine.tif');
>>s=uint8(256*ind2gray(x,map));
>>imshow(s),figure,imshow(s>115 & s<125);
```

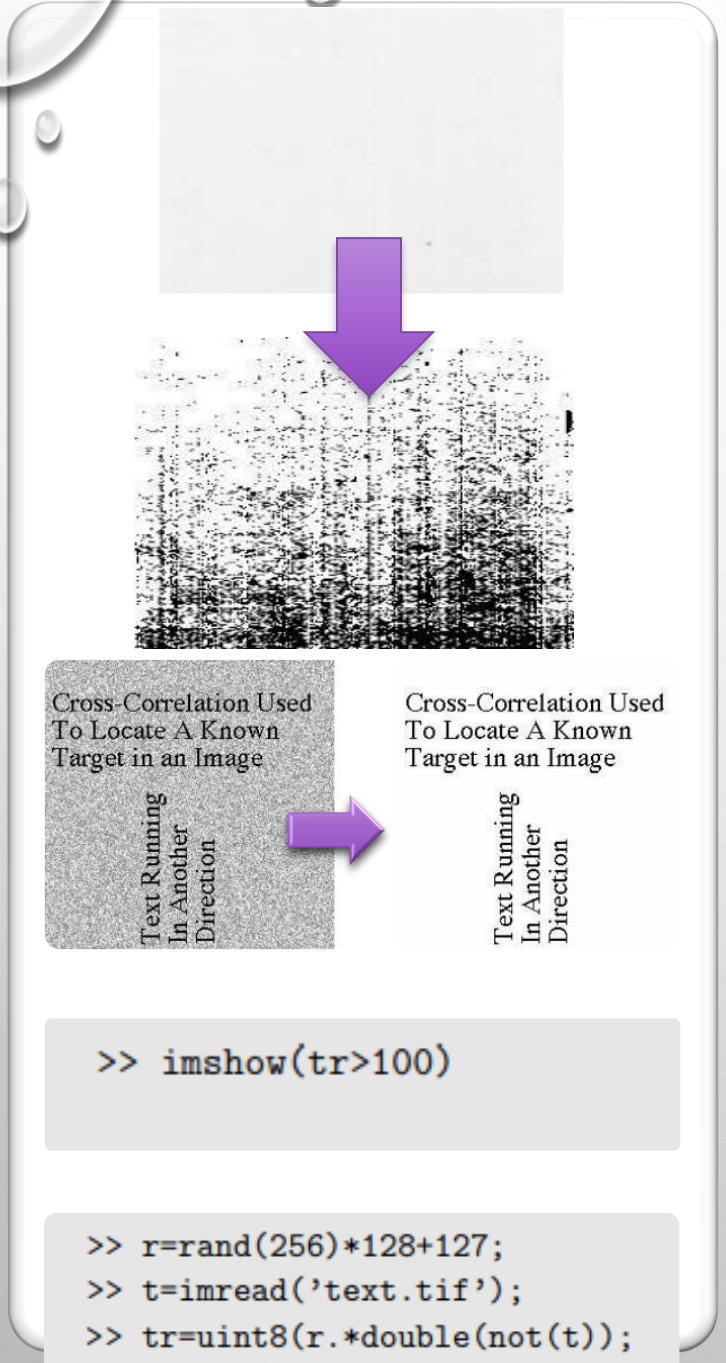




Thresholded image for circles

We can obtain similar results using im2bw

```
imshow(im2bw(x,map,0.45)&~{}im2bw(x,map,0.5))}
```




```
>> imshow(tr>100)
```

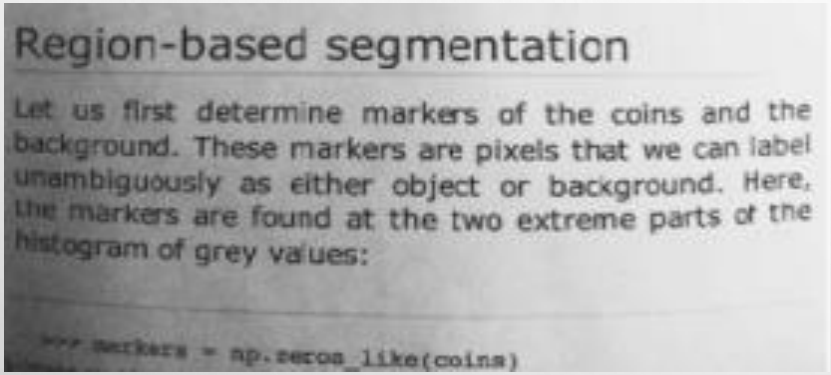
```
>> r=rand(256)*128+127;
>> t=imread('text.tif');
>> tr=uint8(r.*double(not(t));
>> imshow(tr)
```

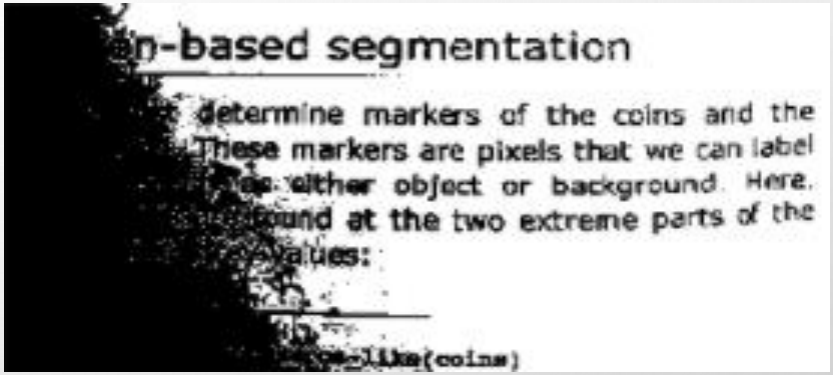
APPLICATIONS OF THRESHOLDING

1. REMOVING UNNECESSARY DETAIL FROM AN IMAGE, TO CONCENTRATE ON ESSENTIALS.
2. BRINGING OUT HIDDEN DETAIL.
3. REMOVING A VARYING BACKGROUND FROM TEXT OR A DRAWING.

```
>> ct=im2bw(e3,t);
```

- NOT ALL OF THE OBJECT HAS BEEN ISOLATED FROM ITS BACKGROUND. EVEN IF DIFFERENT THRESHOLDS ARE USED, THE RESULTS ARE SIMILAR.





An attempt at thresholding

We Can Be Done In Such A Situation Is To Cut The Image Into Small Pieces, And Apply A Threshold To Each Piece Separately.

```
>> p1=c3(:,1:64);
>> p2=c3(:,65:128);
>> p3=c3(:,129:192);
>> p4=c3(:,193:256);
```

1

Now we can threshold each piece:

```
>>g1=im2bw(p1,graythresh(p1));
>>g2=im2bw(p2,graythresh(p2));
>>g3=im2bw(p3,graythresh(p3));
>>g4=im2bw(p4,graythresh(p4));
```

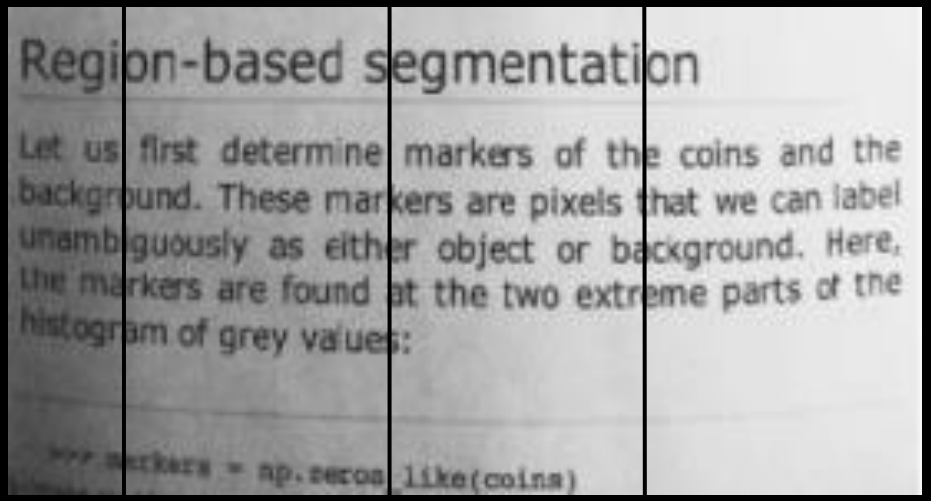
2

To display them as a single image:

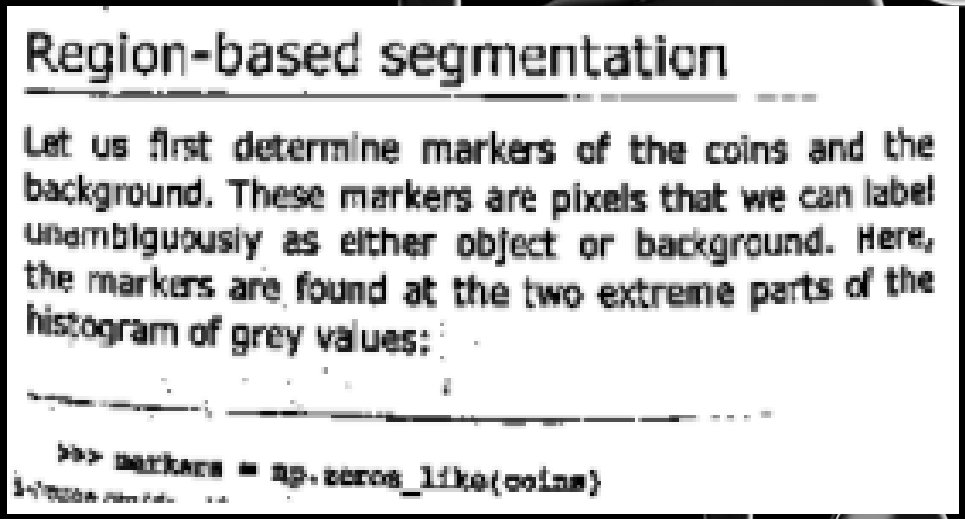
```
>> imshow([g1 g2 g3 g4]);
```

3

ADAPTIVE THRESHOLDING



(a) Cutting up the image



(b) Thresholding each part separately

We can define our function with:

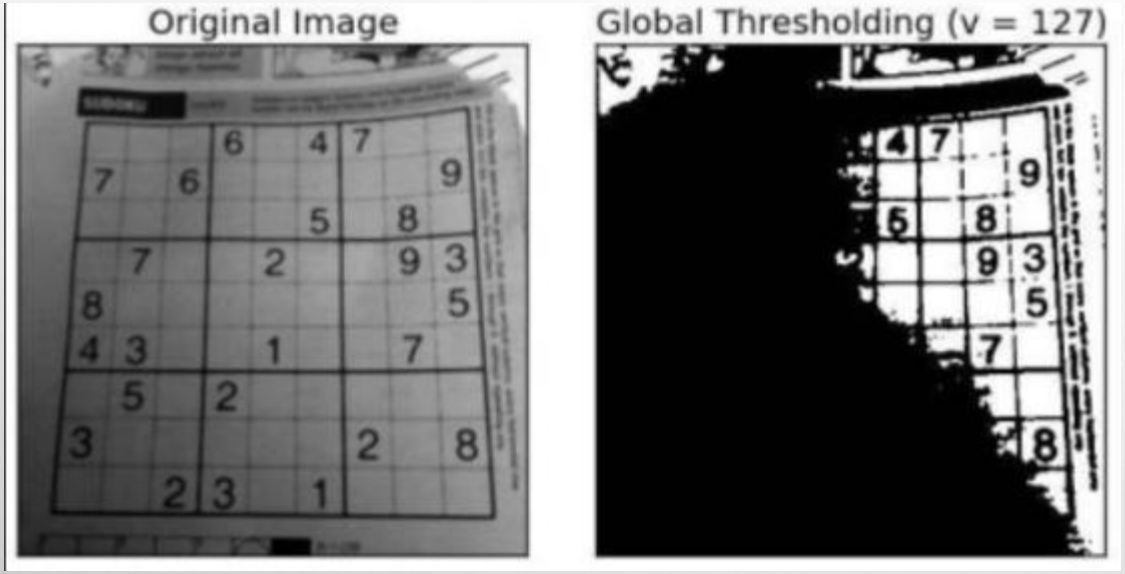
```
>> fun=inline('im2bw(x.graythresh(x))');
```

- Notice that this is the same as the commands used above to create g1, g2, g3 and g4 above, except that now x is used to represent a general input variable.

The function can then be applied it to the image t3 with:

```
>> t4=blkproc(t3,[256,64],fun);
```

EXAMPLE


Original Image
Global Thresholding (v = 127)
Adaptive Mean Thresholding

LOTS OF ISSUES



More than two classes



Automatic choice of *T*



Use of features other than gray value



Imposing “geometry” on the segments (no holes, connected, not too big, …)



Local thresholds rather than global



…

OTSU ADAPTIVE THRESHOLD ALGORITHM

Basic ideas

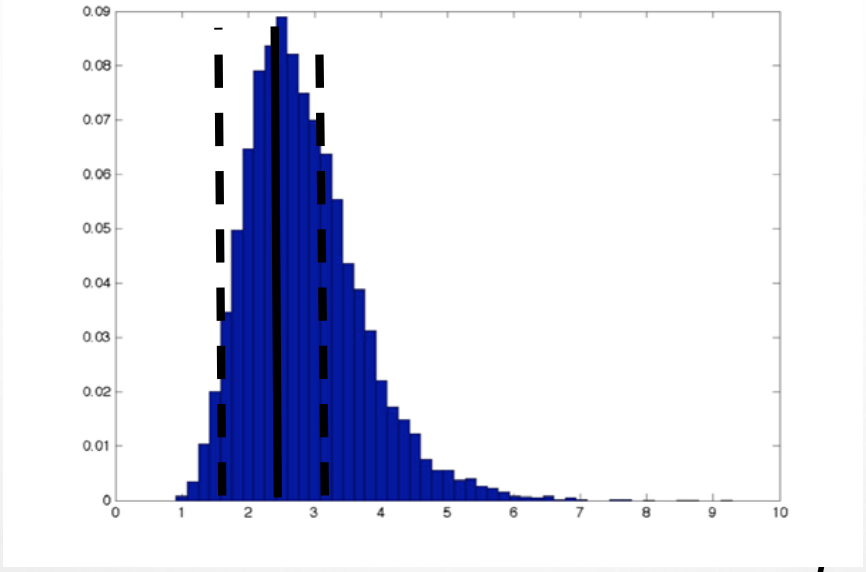
- Separate pixels into two classes
- Want pixels in each class to be as "similar" as possible

Similarity: measured by variance of histogram

- High variance: spread out histogram
- Low variance, very compressed histogram

MEANS AND VARIANCES

p(i)



i

Mean = 2.8
Variance = 0.75
Std Dev = 0.87

Mean = average value of the histogram

$$\mu = \sum_i i p(i)$$

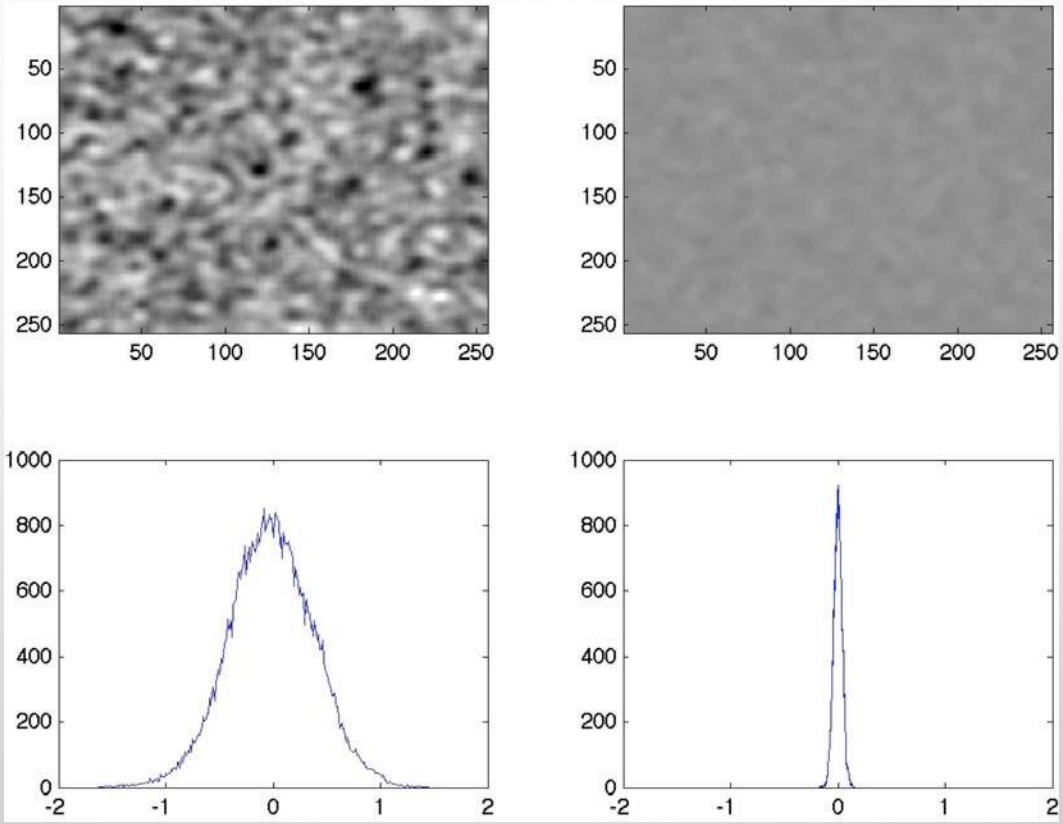
Variance = average spread about the mean

$$\sigma^2 = \sum_i (i - \mu)^2 p(i)$$

Standard deviation = square root of variance

VARIANCES AND HISTOGRAMS

Var = 0.1582 Var = 0.0016



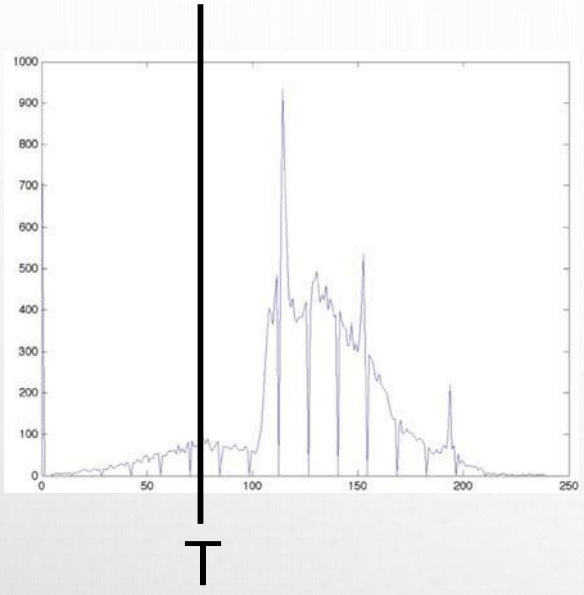
BACK TO OTSU

It is based on a very simple idea: Find the threshold that minimize the weighted within – class variance.

In other words it maximizing the between- class variance.

- Choose threshold to minimize the variance within the two classes
- So, we need to compute the variances of each class as a function of *T*
- Compute the otsu variance for every possible T (e.G., 0, 2, 3, …, 255)
- Choose that threshold for which the otsu variance is minimum
- Exercise in statistics

OTSU STATS



PROBABILITY OF BEING IN CLASS 1 AND 2

$$q_1(T) = \sum_{i=0}^{T} p(i) \quad q_2(T) = \sum_{i=T+1}^{255} p(i)$$

Conditional means of class one and two

$$\mu_1(T) = \sum_{i=0}^{T} i\frac{p(i)}{q_1(T)} \quad \mu_2(T) = \sum_{i=T+1}^{255} i\frac{p(i)}{q_2(T)}$$

Conditional variances of class one and two

$$\sigma_1^2(T) = \sum_{i=0}^{T} (i-\mu_1(T))^2\frac{p(i)}{q_1(T)} \quad \sigma_2^2(T) = \sum_{i=T+1}^{255} (i-\mu_2(T))^2\frac{p(i)}{q_2(T)}$$

Total variance

$$\sigma^2(T) = q_1(T)\sigma_1^2(T) + q_2(T)\sigma_2^2(T)$$

OTSU ADAPTIVE THRESHOLD ALGORITHM

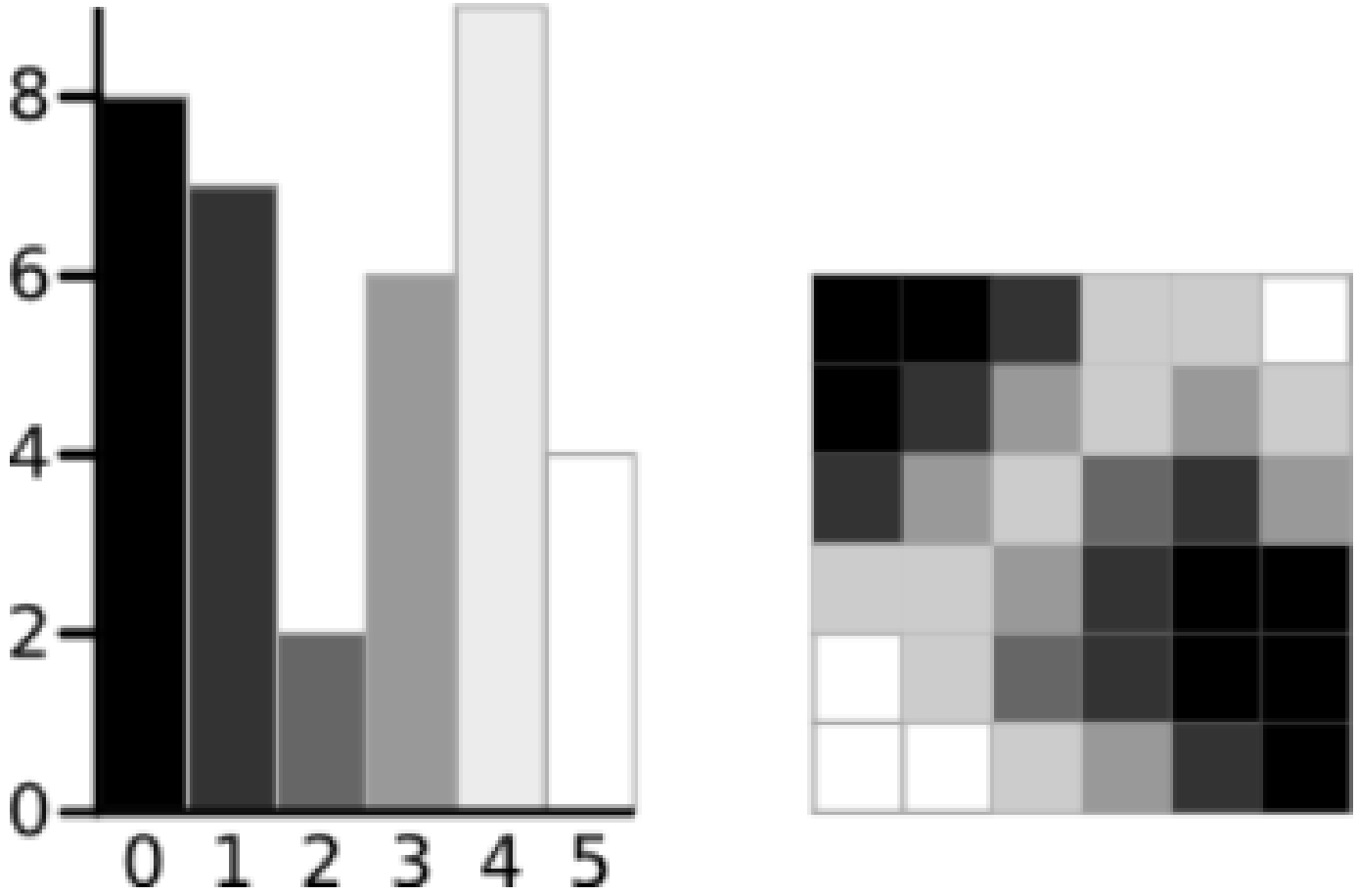
- Converting a greyscale image to monochrome is a common image processing task. Otsu's method, named after its inventor nobuyuki otsu, is one of many binarization algorithms.

Otsu thresholding explained

Otsu's thresholding method involves iterating through all the possible threshold values and calculating a measure of spread for the pixel levels each side of the threshold, i.E. The pixels that either fall in foreground or background. The aim is to find the threshold value where the sum of foreground and background spreads is at its minimum.

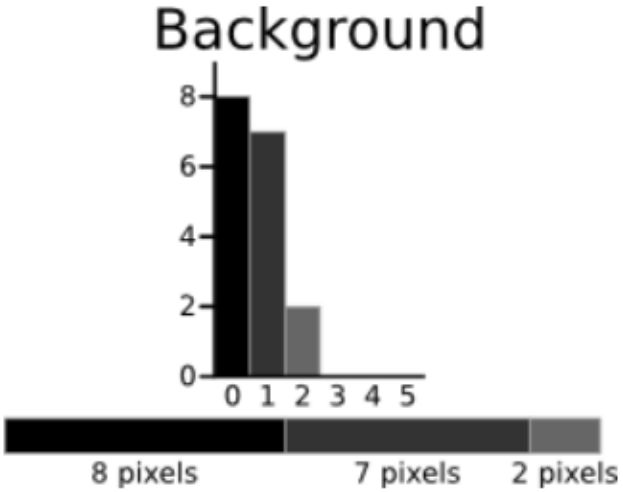
The algorithm will be demonstrated using the simple 6x6 image shown below. The histogram for the image is shown next to it.

To simplify the explanation, only 6 greyscale levels are used.



A 6-level greyscale image and its histogram

The calculations for finding the foreground and background variances (the measure of spread) for a single threshold are now shown. In this case the threshold value is 3



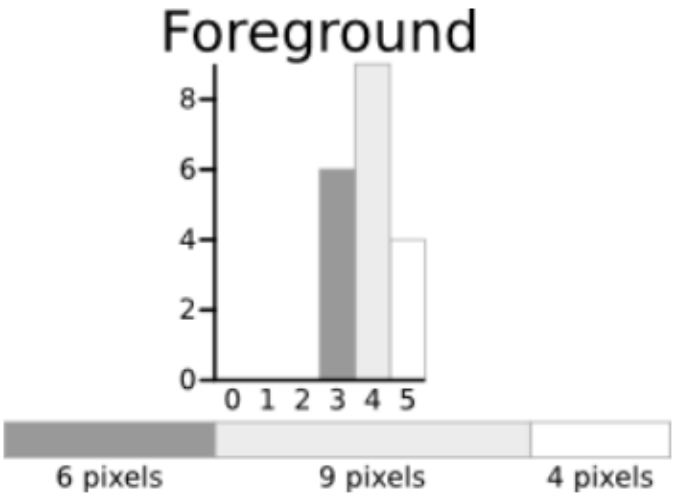
$$\text{Weight} \quad W_b = \frac{8+7+2}{36} = 0.4722$$

$$\text{Mean} \quad \mu_b = \frac{(0\times 8)+(1\times 7)+(2\times 2)}{17} = 0.6471$$

$$\text{Variance} \quad \sigma_b^2 = \frac{((0-0.6471)^2\times 8)+((1-0.6471)^2\times 7)+((2-0.6471)^2\times 2)}{17}$$

$$= \frac{(0.4187\times 8)+(0.1246\times 7)+(1.8304\times 2)}{17}$$

$$= 0.4637$$



$$\text{Weight} \quad W_f = \frac{6+9+4}{36} = 0.5278$$

$$\text{Mean} \quad \mu_f = \frac{(3\times 6)+(4\times 9)+(5\times 4)}{19} = 3.8947$$

$$\text{Variance} \quad \sigma_f^2 = \frac{((3-3.8947)^2\times 6)+((4-3.8947)^2\times 9)+((5-3.8947)^2\times 4)}{19}$$

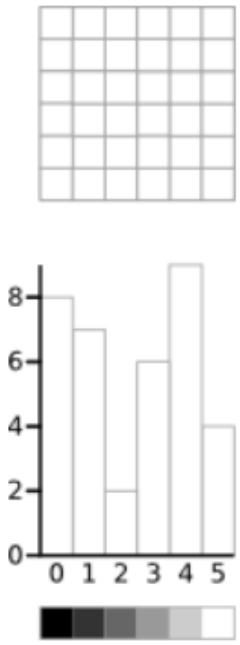
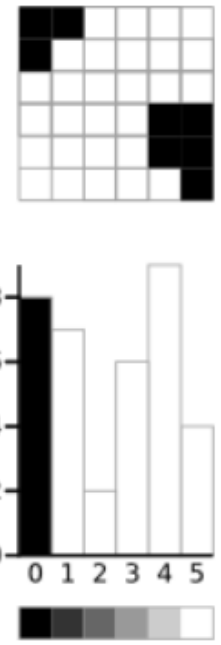
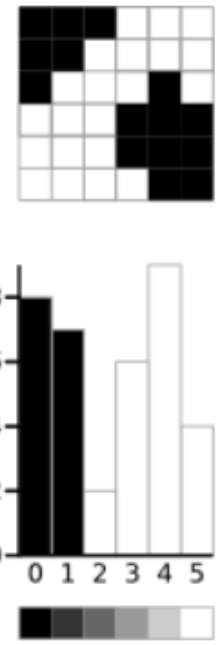
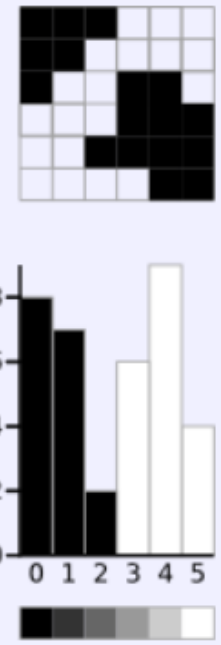
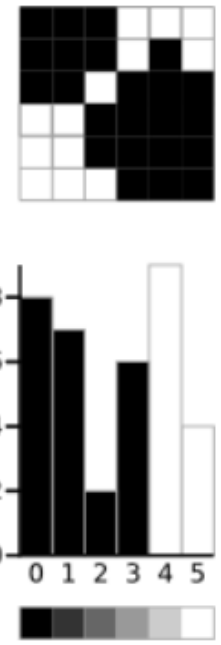
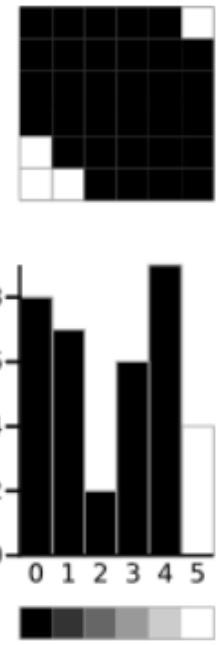
$$= \frac{(4.8033\times 6)+(0.0997\times 9)+(4.8864\times 4)}{19}$$

$$= 0.5152$$

The next step is to calculate the 'within-class variance'. This is simply the sum of the two variances multiplied by their associated weights.

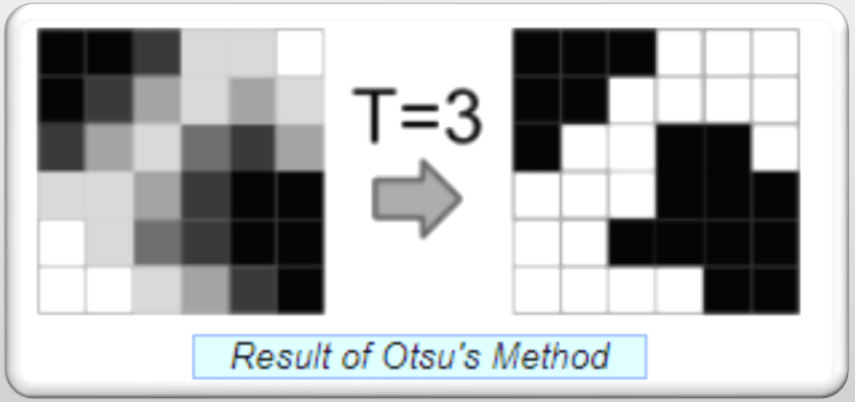
$$\text{Within Class Variance} \quad \sigma_W^2 = W_b\,\sigma_b^2 + W_f\,\sigma_f^2 = 0.4722 * 0.4637 + 0.5278 * 0.5152 = 0.4909$$

This final value is the 'sum of weighted variances' for the threshold value 3. This same calculation needs to be performed for all the possible threshold values 0 to 5. The table below shows the results for these calculations. The highlighted column shows the values for the threshold calculated above.

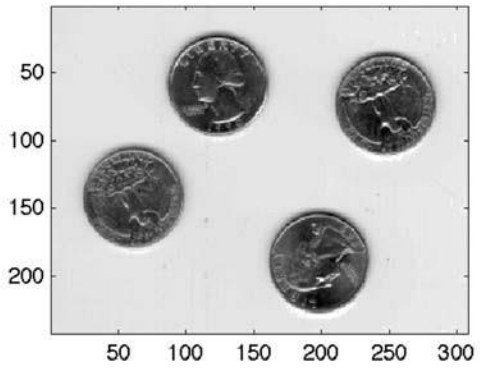
Threshold	T=0	T=1	T=2	T=3	T=4	T=5
						
Weight, Background	$W_b = 0$	$W_b = 0.222$	$W_b = 0.4167$	$W_b = 0.4722$	$W_b = 0.6389$	$W_b = 0.8889$
Mean, Background	$M_b = 0$	$M_b = 0$	$M_b = 0.4667$	$M_b = 0.6471$	$M_b = 1.2609$	$M_b = 2.0313$
Variance, Background	$\sigma^2_b = 0$	$\sigma^2_b = 0$	$\sigma^2_b = 0.2489$	$\sigma^2_b = 0.4637$	$\sigma^2_b = 1.4102$	$\sigma^2_b = 2.5303$
Weight, Foreground	$W_f = 1$	$W_f = 0.7778$	$W_f = 0.5833$	$W_f = 0.5278$	$W_f = 0.3611$	$W_f = 0.1111$
Mean, Foreground	$M_f = 2.3611$	$M_f = 3.0357$	$M_f = 3.7143$	$M_f = 3.8947$	$M_f = 4.3077$	$M_f = 5.000$
Variance, Foreground	$\sigma^2_f = 3.1196$	$\sigma^2_f = 1.9639$	$\sigma^2_f = 0.7755$	$\sigma^2_f = 0.5152$	$\sigma^2_f = 0.2130$	$\sigma^2_f = 0$
Within Class Variance	$\sigma^2_W = 3.1196$	$\sigma^2_W = 1.5268$	$\sigma^2_W = 0.5561$	$\mathbf{\sigma^2_W = 0.4909}$	$\sigma^2_W = 0.9779$	$\sigma^2_W = 2.2491$

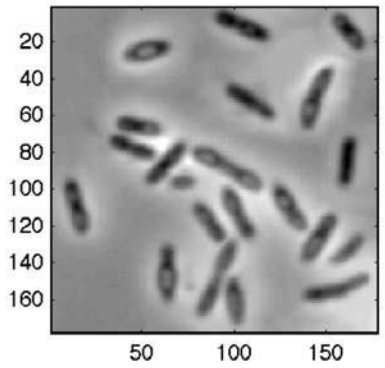
FINALLY

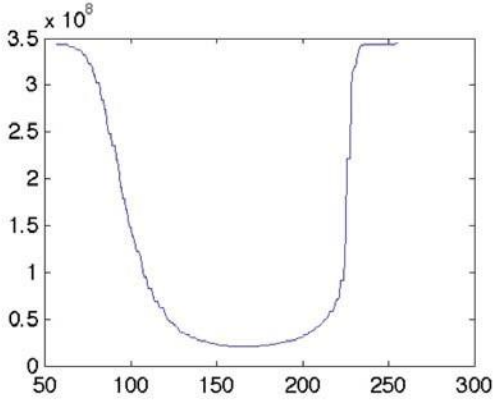
- It can be seen that for the threshold equal to 3, as well as being used for the example, also has the lowest sum of weighted variances. Therefore, this is the final selected threshold. All pixels with a level less than 3 are background, all those with a level equal to or greater than 3 are foreground. As the images in the table show, this threshold works well.

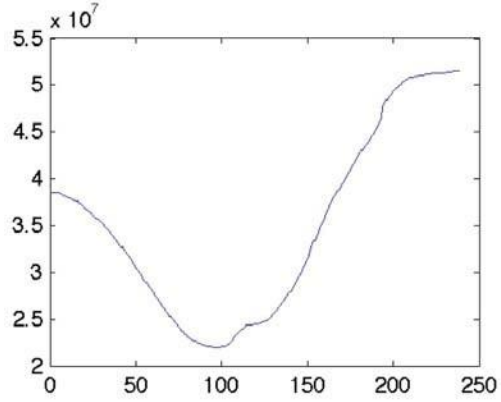


OTSU AT WORK

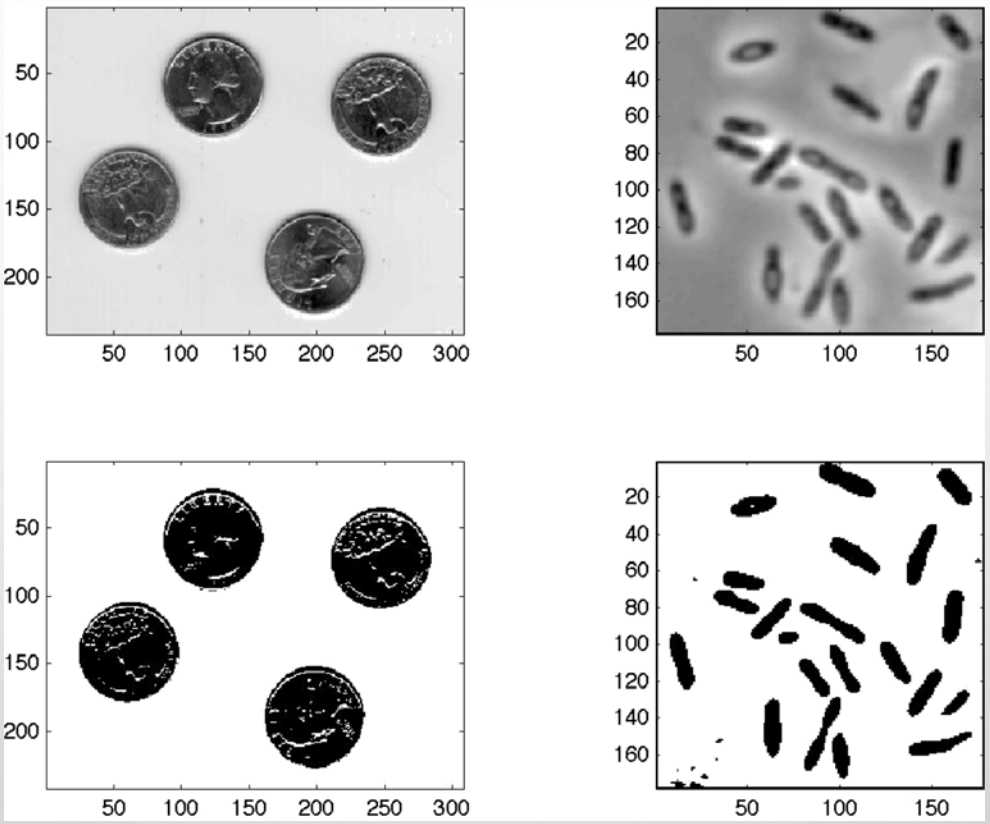

50
100
150
200
50 100 150 200 250 300


20
40
60
80
100
120
140
160
50 100 150

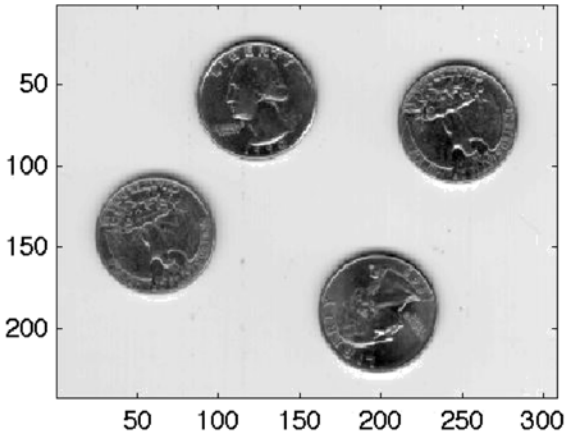

x 10^8
3.5
3
2.5
2
1.5
1
0.5
0
50 100 150 200 250 300

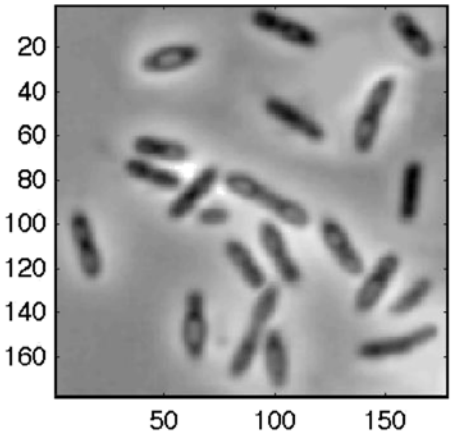

x 10^7
5.5
5
4.5
4
3.5
3
2.5
2
0 50 100 150 200 250

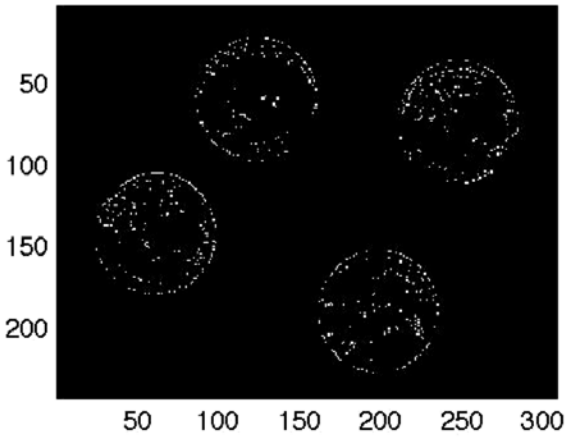
OTSU AT WORK (CONT.)

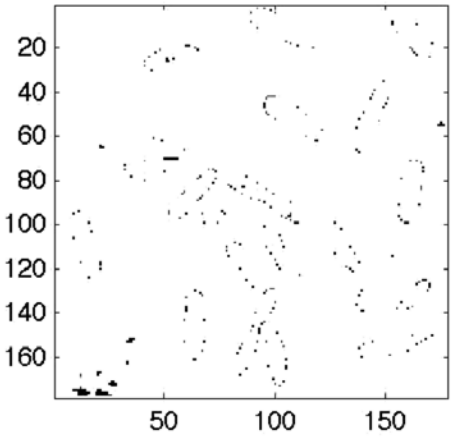


OTSU - MY INITIAL GUESS









MATLAB TOPICS

- Otsu's approach is the basis for the Matlab function **`graythresh`**